

Student 36 **SANGEPU VENKATA PRAVEEN** [192373013RD](#)

2 / 2 submitted

Q1. Smart Hospital Patient Registration System**CODE**

```

import java.util.*;
public class Main {
    static class Patient {
        String patientId;
        String name;
        String mobile;
        String department;
        String doctorName;
        Patient(String patientId, String name, String mobile, String department, String doctorName) {
            this.patientId = patientId;
            this.name = name;
            this.mobile = mobile;
            this.department = department;
            this.doctorName = doctorName;
        }
    }
    static String clean(String s) {
        if (s == null) return "";
        return s.trim().replaceAll("\\s+", " ");
    }
    static boolean isAlphabetic(String s) {
        s = clean(s);
        return s.matches("[A-Za-z ]+");
    }
    static String properCase(String s) {
        s = clean(s);
        String[] parts = s.split(" ");
        StringBuilder sb = new StringBuilder();
        for (String p : parts) {
            if (p.isEmpty()) continue;
            String lower = p.toLowerCase();
            String first = lower.substring(0, 1).toUpperCase();
            String rest = lower.length() > 1 ? lower.substring(1) : "";
            if (sb.length() > 0) sb.append(" ");
            sb.append(first).append(rest);
        }
        return sb.toString();
    }
    static String inputLine(Scanner sc) {
        return clean(sc.nextLine());
    }
    static boolean validateMobile(String mobile) {
        return mobile.matches("\\d{10}");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ArrayList<Patient> patientsList = new ArrayList<>();
        HashMap<String, Patient> patientMap = new HashMap<>();
        HashMap<String, Integer> deptCount = new HashMap<>();
        while (true) {
            String choiceStr = "";
            if (sc.hasNextLine()) choiceStr = clean(sc.nextLine());
            else break;
            if (choiceStr.isEmpty()) continue;
            int choice;
            try {
                choice = Integer.parseInt(choiceStr);
            } catch (Exception e) {
                continue;
            }
        }
    }
}

```

```

    }

    if (choice == 1) {
        if (patientsList.size() >= 100) {

            System.out.println("Maximum patient limit reached.");
            continue;
        }

        String patientId = inputLine(sc);
        if (patientId.isEmpty()) {
            System.out.println("Invalid Patient ID.");
            continue;
        }

        if (patientMap.containsKey(patientId)) {
            System.out.println("Duplicate Patient ID. Registration Failed.");
            continue;
        }

        String name = inputLine(sc);
        if (!isAlphabetic(name)) {
            System.out.println("Invalid Patient Name. Registration Failed.");
            continue;
        }

        String mobile = inputLine(sc);
        if (!validateMobile(mobile)) {
            System.out.println("Invalid Mobile Number. Registration Failed.");
            continue;
        }

        String department = inputLine(sc).toUpperCase();
        department = clean(department);
        String doctorName = inputLine(sc);
        String properName = properCase(name);
        Patient p = new Patient(patientId, properName, mobile, department, doctorName);
        patientsList.add(p);
        patientMap.put(patientId, p);
        deptCount.put(department, deptCount.getOrDefault(department, 0) + 1);
        System.out.println("Patient Registered Successfully.");
    } else if (choice == 2) {
        String patientId = inputLine(sc);
        Patient p = patientMap.get(patientId);
        if (p == null) {
            System.out.println("Patient Not Found");
        } else {
            System.out.println("Patient Details");
            System.out.println("ID : " + p.patientId);
            System.out.println();
            System.out.println("Name : " + p.name);
            System.out.println();
            System.out.println("Mobile : " + p.mobile);
            System.out.println();
            System.out.println("Department : " + p.department);
            System.out.println();
            System.out.println("Doctor : " + p.doctorName);
        }
    } else if (choice == 3) {
        System.out.println("-----");
        System.out.println("ID Name Mobile Department Doctor");
        System.out.println("-----");
        for (Patient p : patientsList) {
            System.out.println(p.patientId + " " + p.name + " " + p.mobile + " " + p.department + " " + p.doctorName);
        }
    } else if (choice == 4) {

```

```
        System.out.println("Department Count");
        ArrayList<String> depts = new ArrayList<>(deptCount.keySet());
        Collections.sort(depts);
        for (String dept : depts) {
            System.out.println();
            System.out.println(dept + " : " + deptCount.get(dept));
        }
    } else if (choice == 5) {
        break;
    }
}
sc.close();
}
```

INPUT

```
1
P101
Rahul Kumar
9876543210
Cardiology
Dr.Arun
1
P102
Priya
9876543211
Ortho
Dr.Meena
3
4
2
P101
5
```

OUTPUT

(no output)

Q2. Online Library Book Management System**CODE**

```
import java.util.*;

class Book {
    String bookId;
    String title;
    String author;
    String category;
    String publisher;
    Book(String bookId, String title, String author, String category, String publisher) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.category = category;
        this.publisher = publisher;
    }
}

public class Main {
    static final int MAX_BOOKS = 500;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> books = new ArrayList<>();
        HashMap<String, Book> bookById = new HashMap<>();
        HashSet<String> uniqueAuthors = new HashSet<>();
        HashMap<String, Integer> categoryCount = new HashMap<>();
        while (true) {
            int choice = readInt(sc);
            switch (choice) {
                case 1:
                    addBook(sc, books, bookById, uniqueAuthors, categoryCount);
                    break;
                case 2:
                    searchBook(sc, bookById);
                    break;
                case 3:
                    displayAllBooks(books);
                    break;
                case 4:
                    displayUniqueAuthors(uniqueAuthors);
                    break;
                case 5:
                    displayCategoryCounts(categoryCount);
                    break;
                case 6:
                    sc.close();
                    return;
                default:
                    break;
            }
        }
    }

    static int readInt(Scanner sc) {
        while (!sc.hasNextInt()) {
            sc.next();
        }
        return sc.nextInt();
    }

    static void addBook(Scanner sc, ArrayList<Book> books, HashMap<String, Book> bookById,
        HashSet<String> uniqueAuthors, HashMap<String, Integer> categoryCount) {

        if (books.size() >= MAX_BOOKS) return;
        sc.nextLine();
```

```

String bookId = readNonEmptyLine(sc, "");
bookId = bookId.trim();
if (bookById.containsKey(bookId)) {
    return;
}
String bookTitle = readNonEmptyLine(sc, "");
bookTitle = bookTitle.trim();
bookTitle = toTitleCase(bookTitle);

String authorName = readNonEmptyLine(sc, "");
authorName = authorName.trim();

String category = readNonEmptyLine(sc, "");
category = category.trim().toUpperCase();

String publisher = readNonEmptyLine(sc, "");
publisher = publisher.trim();

if (publisher.isEmpty()) {
    return;
}

Book b = new Book(bookId, bookTitle, authorName, category, publisher);

books.add(b);
bookById.put(bookId, b);
uniqueAuthors.add(authorName);

categoryCount.put(category, categoryCount.getOrDefault(category, 0) + 1);
System.out.println("Book Added Successfully.");
}

static void searchBook(Scanner sc, HashMap<String, Book> bookById) {
    sc.nextLine();
    String id = readNonEmptyLine(sc, "").trim();
    Book b = bookById.get(id);
    if (b != null) {
        System.out.println(b.bookId + " " + b.title + " " + b.author + " " + b.category + " " + b.publisher);
    } else {
        System.out.println("Book Not Available");
    }
}

static void displayAllBooks(ArrayList<Book> books) {
    System.out.println("-----");
    System.out.println("ID TITLE AUTHOR CATEGORY");
    System.out.println("-----");
    for (Book b : books) {
        System.out.println(b.bookId + " " + b.title + " " + b.author + " " + b.category);
    }
}

static void displayUniqueAuthors(HashSet<String> uniqueAuthors) {
    System.out.println("Unique Authors");
    ArrayList<String> list = new ArrayList<>(uniqueAuthors);
    Collections.sort(list);
    for (String a : list) {
        System.out.println(a);
    }
}

static void displayCategoryCounts(HashMap<String, Integer> categoryCount) {
    System.out.println("Category-wise Count");
    ArrayList<String> cats = new ArrayList<>(categoryCount.keySet());
    Collections.sort(cats);
    for (String c : cats) {
        System.out.println(c + " : " + categoryCount.get(c));
    }
}

```

```
}
static String readNonEmptyLine(Scanner sc, String promptIgnored) {
    String line = sc.nextLine();
    while (line != null && line.isEmpty()) {
        line = sc.nextLine();
    }
    return line;
}
static String toTitleCase(String s) {
    String[] parts = s.trim().toLowerCase().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < parts.length; i++) {
        if (parts[i].length() == 0) continue;
        sb.append(Character.toUpperCase(parts[i].charAt(0)));
        if (parts[i].length() > 1) sb.append(parts[i].substring(1));
        if (i != parts.length - 1) sb.append(" ");
    }
    return sb.toString().trim();
}
}
```

INPUT

```
1
B101
java programming
Herbert Schildt
Programming
McGraw Hill
1
B102
data structures
Mark Allen
Computer Science
Pearson
3
4
5
6
```

OUTPUT

(no output)

Student 37 **Sareddy Varun Ram Kumar Reddy** 192565023RD

0 / 2 submitted

Q1. Smart Hospital Patient Registration System

— Not Submitted —

Q2. Online Library Book Management System

— Not Submitted —